Learning from Physical Experience: Context-Space Adaptation for Robotic Manipulation

Yihan Zeng[1,2,3], **Yisu Li**[1], **Kaixin Huang**[1], **Marc WuDunn**[2], **Amelia Kovacs**[2], **Mikael Harseno**[2], **Kimmy Lin**[2], **Porter Hunley**[2], **Fred Shentu**[2], **Philipp Wu**[2], **Zhongyu Li**[1], **Zike Yan**[1]

[1]The Chinese University of Hong Kong, [2]xdof.ai, [3]University of Electronic Science and Technology of China

Abstract

Recent multimodal foundation models exhibit strong perception, reasoning, and in-context learning capabilities, but translating these capabilities into physical action remains challenging. Through real-world experiments across diverse manipulation tasks, we find that ChatGPT-6-Astra, a state-of-the-art general-purpose reasoner, can serve as a physically actionable, context-responsive manipulation policy, demonstrating effective fine manipulation and spatial reasoning. Building on these findings, we introduce LEXP, a harness for learning from physical experience through context-space adaptation. LEXP combines task-level planning with an experience module that interprets execution evidence, extracts and retains lessons with applicability conditions, and reuses relevant lessons to generate contextual feedback for subsequent actions. Model weights and the action interface remain fixed throughout. Across 12 real-world manipulation tasks, LEXP achieves 23/36 successful trials (63.9%), compared with 19/36 (52.8%) with memory alone and 2/36 (5.6%) for vanilla Astra. On three tasks, bread transfer, sort trash and toiletries, and retrieve drawer, removing planning reduces success from 7/9 to 4/9, while removing retained experience reduces it to 3/9. Execution traces illustrate feedback-guided corrections and cross-episode reuse of an inspection strategy. Execution analyses illustrate how task structure and experience-derived feedback guide task progression, action correction, and experience reuse. These results support context-space adaptation as a practical mechanism for turning physical interaction into reusable guidance for robotic manipulation.

Resources: Project Page

1 Introduction

A long-standing goal of robotic manipulation is to perform diverse tasks reliably in unfamiliar and changing physical environments. This requires robots to adapt when familiar strategies fail and reuse what they learn from physical interaction. The challenge is to turn limited physical experience into reliable, transferable improvements in behavior. Existing approaches pursue this goal through different adaptation mechanisms. Vision–language–action (VLA) models learn from robot demonstrations through supervised fine-tuning [10, 22]. World action models [29]aim to reduce task-specific adaptation by jointly predicting video and actions, enabling zero-shot execution of unseen tasks and environments on pretrained embodiments, while related video-model policies [11] instead use demonstration-based fine-tuning and rollout-informed planning . Real-world reinforcement learning improves policy parameters through repeated physical interaction

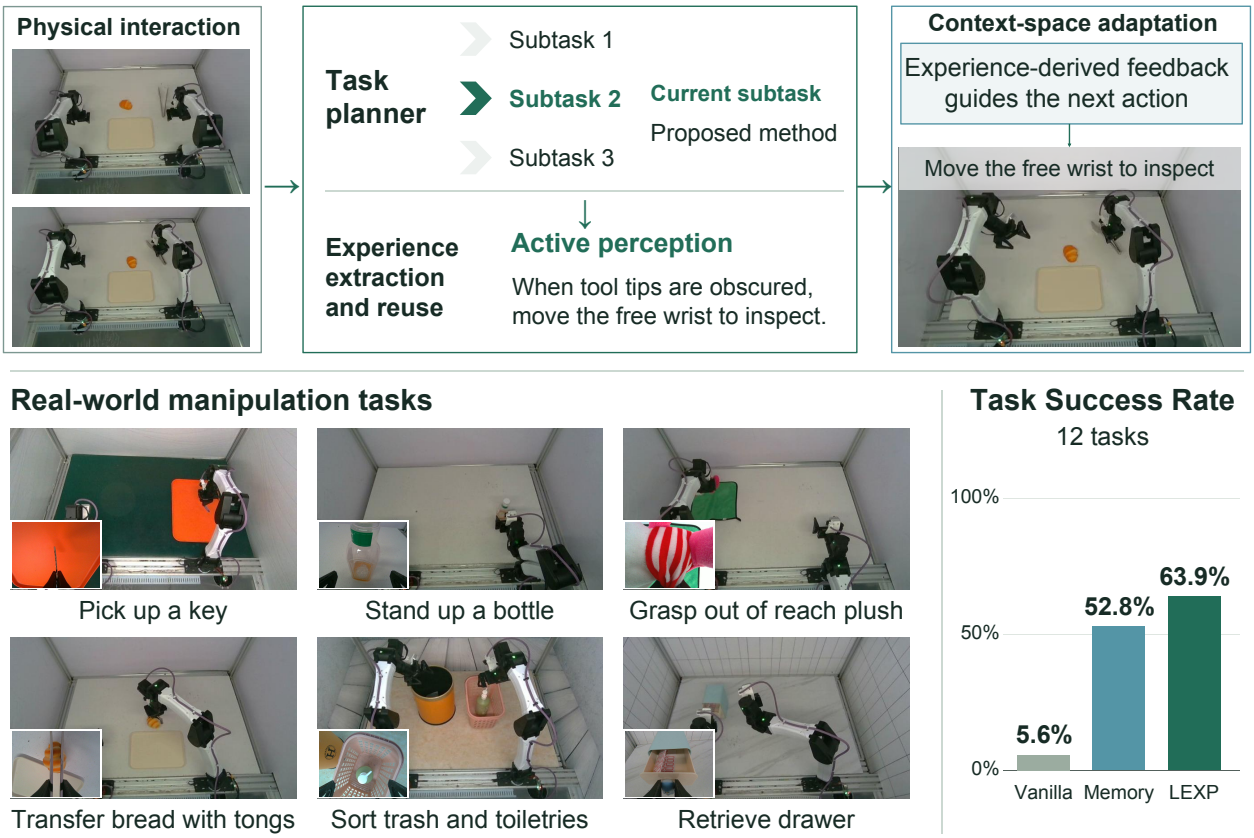


Figure 1 Learning from physical experience through context-space adaptation. LEXP combines task-level planning with experience extraction and reuse to guide subsequent actions while keeping model weights and the action interface fixed. Across 12 real-world manipulation tasks, LEXP achieves 63.9% success, compared with 52.8% with memory alone and 5.6% for vanilla Astra.

[21], but requires reward specification and infrastructure for physical trials and resets [17, 18, 21]. These requirements make demonstrations, reward engineering and physical trial infrastructure central to the cost of adaptation. General-purpose multimodal reasoners offer a complementary interface: observations, instructions and execution feedback can condition executable tool calls without updating model weights. Their relevance to robotics depends on whether this contextual flexibility supports effective action revision and reusable improvements under a limited physical interaction budget.

We investigate this possibility through real-world experiments with GPT-6 Astra [20], across 12 robotic manipulation tasks covering fine manipulation, spatial reasoning, long-horizon execution, tool use, bimanual coordination, and force control. Across these tasks, we vary access to interaction history, and examine recovery demonstrations and online language guidance. Two complementary findings emerge. First, Astra shows promising fine-manipulation and spatial reasoning capabilities when given access to interaction history: interaction history inform subsequent corrections and help the robot reason about object relations and pose. Second, its behavior is steerable through context. Visual demonstrations and textual corrections support revisions to grasp checking, object repositioning, motion direction, arm selection, and action order. These benefits vary across tasks, but together highlight both Astra's executable competence and its capacity for context steering—two prerequisites for improving manipulation through experience-derived context rather than additional parameter training.

Building on these observations,we introduce LEXP, a harness for learning from physical experience through context-space adaptation (Figure 1). A context compiler combines task instructions, current observations, interaction history, and agent-generated experience feedback to condition executable actions. A planner

decomposes the task into subtasks, tracks progress, and proposes a method for the current subtask. The experience module evaluates this method against recent actions, measured execution outcomes, and current observations, drawing on retained lessons to generate feedback for the actor's next decision. It also extracts and updates structured lessons from physical interaction, recording their applicability, supporting evidence, and limitations for subsequent reuse. This separates interaction history, which preserves past exchanges, from reusable experience, which retains findings beyond the history window and across attempts. The models remain frozen and the action interface stays fixed; adaptation occurs through changes to the context guiding execution. The design builds on reflective context optimization [1, 3] and prior physical adaptation systems [6, 16, 28, 34].

Across 12 real-world manipulation tasks, LEXP achieves 23/36 successful trials, compared with 19/36 with memory alone and 2/36 for vanilla Astra. Ablations and execution analyses further examine how task-level planning and retained experience support task progression and subsequent action decisions.

Our contributions are threefold:

- A real-world evaluation suite spanning 12 manipulation tasks to investigate executable capabilities, adaptation through interaction history, and context steering in a general-purpose multimodal reasoner. Across three fine-manipulation tasks, memory increases Astra's success rate from 0/9 to 8/9.
- LEXP, a harness that combines task-level planning with the extraction and reuse of physical experience to guide robotic manipulation through context-space adaptation. Across 12 tasks, LEXP achieves 23/36 successful trials, compared with 19/36 with memory alone and 2/36 for vanilla Astra.
- Component ablations and execution analyses examining the roles of planning and retained experience. Across three tasks, bread transfer, sort trash and toiletries, and retrieve drawer, complete LEXP achieves 7/9 successful trials, compared with 4/9 without planning and 3/9 without retained experience.

2 Problem Statement

2.1 Objective: improve policy behavior through reusable experience

Our objective is to improve robotic manipulation by converting physical experience into reusable guidance for subsequent behavior. Supervised fine-tuning encodes demonstrated actions in model parameters, while reinforcement learning adjusts a policy using rewards from interaction. We pursue the same practical goal to improve performance but move the variables being optimized from model parameters to the context that conditions the policy. With model weights and the robot interface fixed, contextual guidance can change how the actor advances the current subtask, approaches an object, verifies a grasp, and responds to failure.

Context-space Adaptation. Let π_θ denote a pretrained actor with fixed parameters θ, and let $\mathcal{I}$ denote its fixed robot-control interface. Given a task instruction l and a current observation o_t, the actor selects an action conditioned on a compiled context:

$$a_t \sim \pi_\theta(\cdot \mid c_t), \qquad c_t = \mathcal{C}(l, o_t, H_t, f_t), \tag{1}$$

where H_t is memory consisting of interaction history, f_t is contextual guidance, and $\mathcal{C}$ is the context compiler. Executing a_t through $\mathcal{I}$ produces an execution record r_t and a subsequent observation o_{t+1}. The resulting evidence $e_t = (o_t, a_t, r_t, o_{t+1})$ records the proposed action, its execution result, and the observed physical consequences.

We implement this adaptation through structured, modular context updates [32]. After each interaction, new evidence is appended to memory: $H_{t+1} = \text{Append}(H_t, e_t)$. Let M_t denote retained experience and p_{t+1} denote the planner's current subtask and proposed method. The experience module $\mathcal{E}$ uses this task-level reference together with memory, the current observation, and retained experience to update lessons and generate feedback:

$$\begin{aligned} (M_{t+1}, f_{t+1}) &= \mathcal{E}(o_{t+1}, H_{t+1}, p_{t+1}, M_t), \\ c_{t+1} &= \mathcal{C}(l, o_{t+1}, H_{t+1}, f_{t+1}). \end{aligned} \tag{2}$$

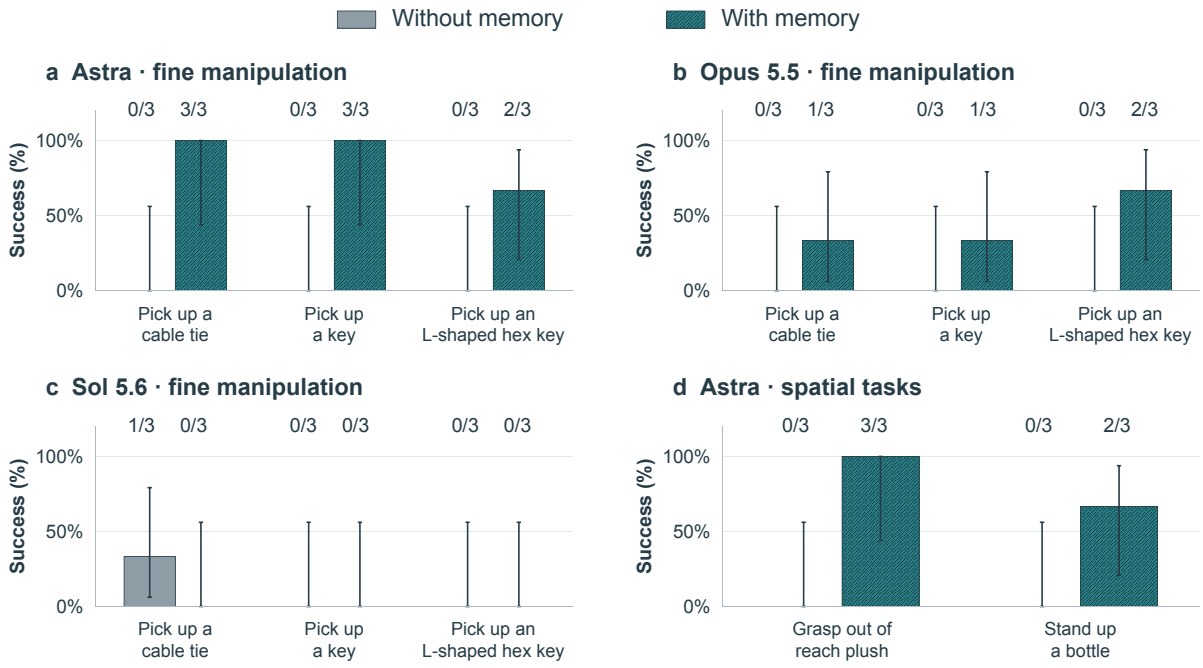


Figure 2 Memory supports fine manipulation and spatial reasoning. Success rates with and without memory for (a) GPT-6 Astra, (b) Opus 5.5, and (c) Sol 5.6 on three fine-manipulation tasks, and (d) GPT-6 Astra on two tasks requiring spatial reasoning. Memory consists of prior observations, actions and intention notes, and execution outcomes retained in the actor's context. Bars show observed success rates, with successful trials over total trials indicated above each bar; error bars denote 95% Wilson confidence intervals. Memory improves Astra's performance across the evaluated tasks.

Planning and retained experience thus inform the actor through generated feedback. Physical execution supplies new evidence, which updates memory, retained experience, and feedback; the context compiler then assembles the next actor input, closing the adaptation loop. Retained experience remains available across attempts, allowing lessons to be reused beyond their source interactions. Module-specific prompts are provided in Appendix E.

2.2 Preliminary capability and context-use evidence of GPT-6 Astra

We investigate the prerequisites for context-space adaptation through real-world experiments with GPT-6 Astra [20] across 12 robotic manipulation tasks covering fine manipulation, spatial reasoning, long-horizon execution, tool use, bimanual coordination, and force control. We compare execution with and without interaction history and separately examine the effects of recovery demonstrations and online language feedback. These experiments examine two empirical prerequisites for context-space adaptation: whether memory enables the actor to use physical interaction evidence in subsequent decisions, and whether contextual guidance can steer its behavior.

Memory benefits fine manipulation through outcome-aware corrections. Figure 2a compares Astra's small-object pickup performance with and without memory. Overall success rises from 0/9 (0%) without memory to 8/9 (88.9%) with memory. The task-level contrasts are 0/3 versus 3/3 for cable ties, 0/3 versus 3/3 for keys, and 0/3 versus 2/3 for the hex key. Here, memory consists of interaction history carried into the actor's context, including previous observations, actions and their intention notes, and execution outcomes.

These results show that Astra can execute precision-demanding grasps and that memory guidance can help it deploy this capability successfully which allows the actor to interpret the current scene in relation to earlier attempts, assess what an action changed, and use that evidence to revise the next command. Poor performance

without memory therefore does not imply that the required manipulation is beyond its execution capability. Instead, the contrasts motivate studying how prior observations and action outcomes guide subsequent choices. This benefit is actor-dependent: across the same three tasks, Opus 5.5 improves from 0/9 to 4/9, whereas Sol 5.6 changes from 1/9 to 0/9 (Figure 2b–c). The observations support experience-guided adaptation, while leaving open which history contents produce the gains.

Memory supports spatial reasoning about object relations and pose. Across two spatial manipulation tasks, Astra's observed success increases from 0/3 without memory to 3/3 with memory for grasping an out-of-reach plush, and from 0/3 to 2/3 for standing a fallen bottle upright. The plush task involves reasoning about reachability and object relations, while bottle reorientation requires tracking changes in orientation and support. By retaining previous observations, actions, and execution outcomes, memory makes the object's response to manipulation available to subsequent decisions, helping the actor interpret spatial configurations through interaction rather than appearance alone.

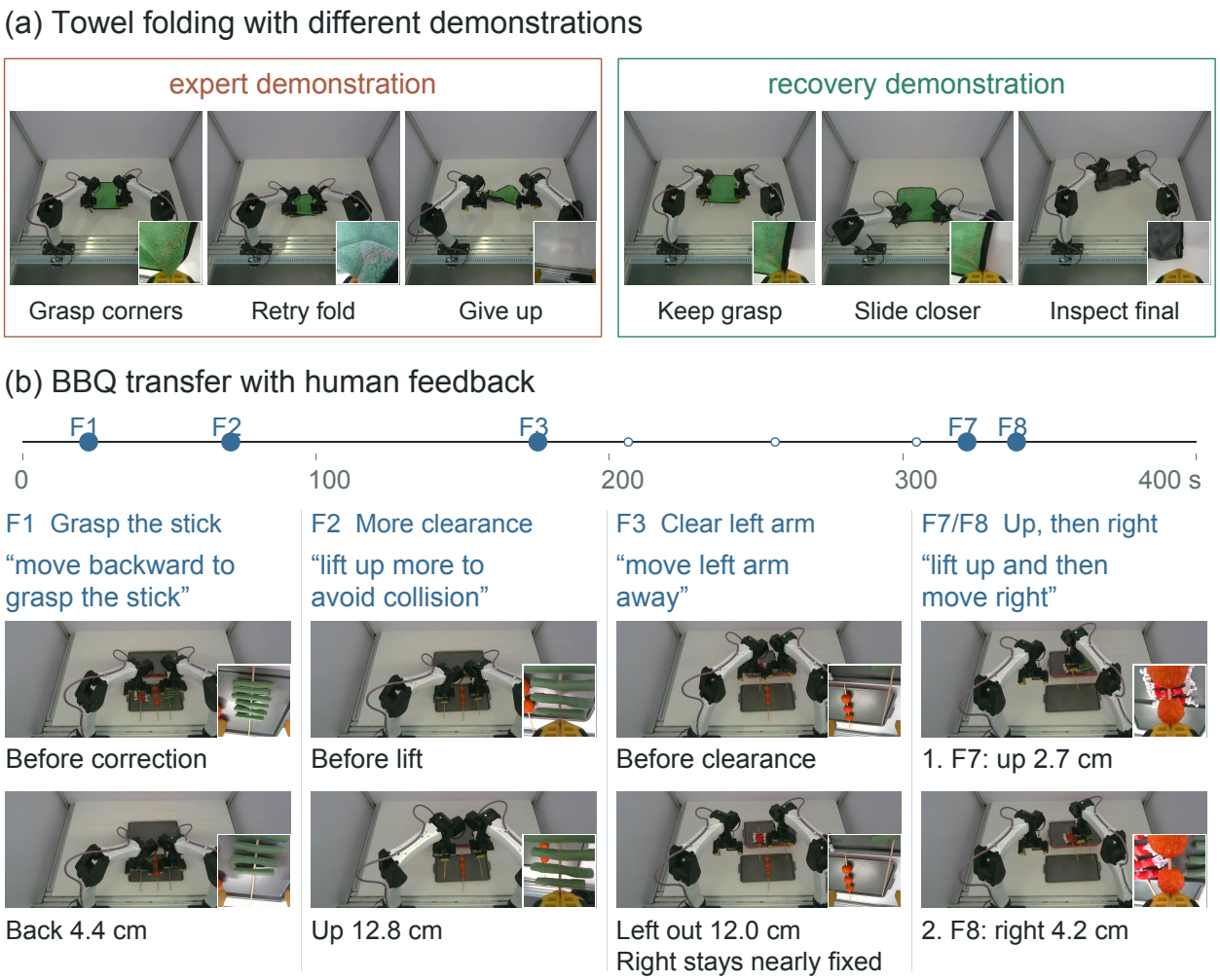


Figure 3 Context steering changes strategies and follows spatially specific instructions. (a) Expert and recovery demonstrations lead to different grasp-checking and repositioning strategies during towel folding. (b) During barbecue transfer, online language feedback guides changes in grasp approach, lifting clearance, arm selection, and motion order. Selected observations and measured displacements illustrate the robot's responses, including moving backward to grasp, lifting to avoid collision, moving the left arm away, and lifting before moving right. The timeline marks feedback delivery; F labels identify individual feedback messages.

Context steering changes strategies and follows spatially specific instructions. Visual demonstrations and

online language feedback steer behavior at both the strategy and motion levels. In towel folding, a clean demonstration leads Astra to grasp the near corners, retry the fold, and eventually give up, whereas a recovery demonstration leads it to verify edge capture, maintain the grasp, bring the towel closer, and inspect the resulting fold (Figure 3a). The difference concerns grasp verification and workspace management, showing that demonstration context can redirect the manipulation strategy.

Figure 3b illustrates online instruction following during barbecue transfer. A request to move backward is followed by a 4.4 cm displacement; feedback requesting more clearance produces a 12.8 cm lift; and guidance to clear the left arm produces a 12.0 cm left-arm motion while the right remains fixed. "Up, then right" guidance leads to a 2.7 cm rise followed by a 4.2 cm lateral motion. These examples demonstrate responsiveness to motion direction, arm selection, and action order. Visual demonstration context and textual feedback thus provide concrete channels for revising both high-level strategies and individual actions, motivating our harness design to deliver guidance derived from physical experience through context.

From context use to experience-derived adaptation. These observations motivate a harness that turns physical interaction into reusable contextual guidance. Memory makes past interactions available, while demonstrations and language feedback show that context can steer subsequent behavior. The next question is whether useful guidance can be extracted from the actor's own successful and failed interactions and reused in new executions.

3 Method

Building on these observations, we introduce LEXP, a harness for learning from physical experience through context-space adaptation. As shown in Figure 4 , LEXP combines executable action decisions, task-level planning, and experience-derived feedback to turn physical interactions into reusable guidance, while keeping model weights and the action interface fixed. We first present the framework and its information flow in Section 3.1. We then describe how the actor's context is constructed and its actions are executed in Section 3.2, followed by task decomposition and progress tracking in Section 3.3. Finally, Section 3.4 details how physical experience is extracted, retained, and reused to generate feedback for subsequent actions.

3.1 Overview

Figure 4 illustrates LEXP's interaction loop. At each decision, a context compiler assembles the system prompt, task instruction, current visual observations and robot states, memory, and available guidance for the actor. The actor issues a tool call specifying an action and an accompanying intention note. The execution layer checks the proposed action before execution and records its outcome. Subsequent observations capture the resulting robot state and scene, completing an interaction record for later decisions.

Alongside this loop, the planner decomposes the task into subtasks, assesses progress from the observed scene, and proposes a method for the current subtask. These outputs inform the experience module. The experience module checks the proposed method against recent interactions and current observations, applies relevant retained lessons, and generates feedback for the actor's next action. It also extracts and revises lessons as new evidence becomes available, preserving their applicability conditions, supporting evidence, and limitations. Physical interaction thus informs future behavior through both raw history and reusable experience, with the latter translated into contextual feedback for the current decision.

3.2 Actor Context and Action Execution

The context compiler assembles the actor's input from the system prompt, task instruction, current camera observations and robot states, memory and available guidance. Memory contains the proposed action and its intention note, execution outcome, and observations before and after the action, allowing the actor to relate the current scene to previous attempts. The planner's proposed method and retained experience are interpreted by the experience module and reach the actor through its feedback.

At each decision, the actor produces one tool call containing a motion command and an intention note, or signals completion or termination. The execution layer validates proposed motions against the latest robot state and execution constraints, then executes accepted commands. The command, execution outcome, and

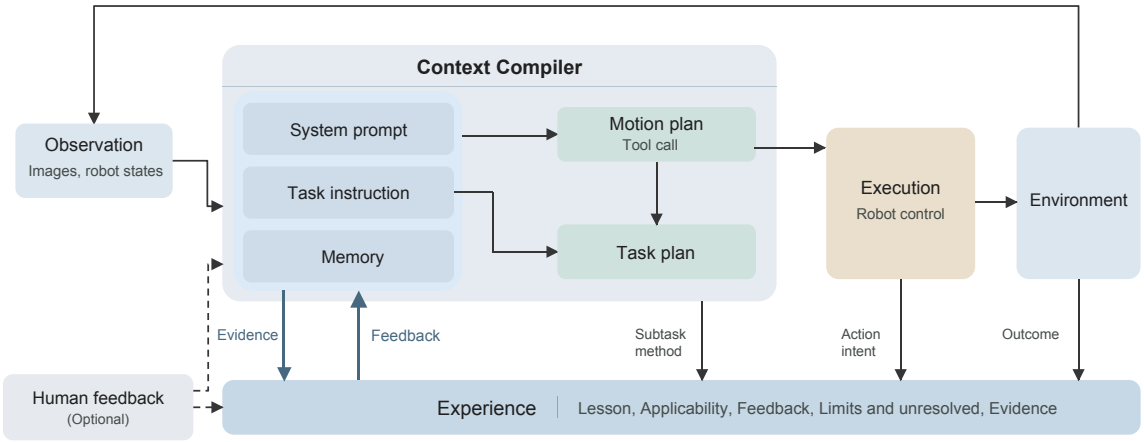


Figure 4 Method overview. The context compiler combines task instructions, current observations and robot states, memory, and experience-derived feedback to condition the actor's next action. Physical execution produces outcomes and subsequent observations that become evidence for later decisions. The planner tracks task progress and proposes a method for the current subtask. The experience module interprets this task-level structure alongside physical evidence and retained lessons, generating contextual feedback while extracting and revising reusable experience. Model weights and the action interface remain fixed throughout.

subsequent observations are recorded as a completed interaction. These records provide evidence of the action's physical effects for both the actor's history and the experience module.

3.3 Task-Level Planning

Planning provides task-level structure for reasoning and execution. Individual action decisions must serve the overall task, while retained lessons must be interpreted in relation to what the robot is currently trying to accomplish. The planner supplies this structure by decomposing the task into an ordered list of subtasks. As execution proceeds, it assesses progress against the observed scene, identifies the current subtask, and proposes a method for carrying it out. Together, the current subtask and proposed method give the experience module a reference for assessing recent execution and applying relevant lessons to the current situation. Through the resulting feedback, planning guides task-level reasoning and the application of experience.

3.4 From Physical Experience to Context Space Adaptation

The experience module turns physical interaction into contextual guidance by combining recent execution evidence with retained lessons. It receives the current subtask and proposed method from the planner, recent interaction records, and current observations. It first checks whether the proposed method's condition is satisfied and compares observed execution with the method. This assessment grounds feedback in what the robot commanded and what the observations show, rather than assuming that an accepted command achieved its intended physical effect.

Extracting and retaining experience. Alongside feedback generation, the module records useful findings from completed interactions. These may include observed object relations, attempted methods and their results, partial successes, unsuccessful attempts, and execution constraints. Each lesson is retained with its applicability conditions, its potential use in later decisions, supporting interaction records, and limitations or unresolved explanations. New evidence can revise an existing lesson, allowing retained experience to change as the robot encounters additional physical evidence. Unlike raw history, these entries preserve selected findings after the original interactions leave the actor's history window.

Reusing experience through contextual feedback. For subsequent decisions, the module considers lessons retained from the current attempt or carried over from earlier attempts. It evaluates their applicability against the current observations, using the planner's subtask and proposed method as a task-level reference. Relevant lessons inform a concise feedback message supplied to the actor before its next action; when retained experience conflicts with current observations, the observations take precedence. Experience reuse therefore involves interpreting prior findings in the current physical situation and translating them into actionable contextual guidance.

4 Experiments

We evaluate LEXP through real-world manipulation experiments organized around four questions: **Q1**, Does LEXP improve performance on real world manipulation tasks? **Q2**, How does task-level planning guide the actor's reasoning and task progression? **Q3**, Can lessons extracted from physical interaction guide later decisions beyond the interactions from which they were learned? **Q4**, How does experience-derived feedback shape the actor's action decisions?

4.1 Experimental Setup

Platform and tasks. We conduct experiments on a bimanual YAM robot equipped with two grippers and three view realsense cameras. The actor receives camera observations and robot states and controls the robot through a tool-call interface. Our evaluation covers 12 manipulation tasks spanning fine manipulation, spatial reasoning, long-horizon execution, tool use, bimanual coordination, and force control. Figure 5 illustrates the 12 tasks and representative stages of their execution. Task details are provided in Appendix A. The preliminary experiments in Section 2.2 characterize memory and context steering; here, we evaluate LEXP's manipulation performance and examine the roles of task-level planning and experience reuse.

Comparison conditions. We compare the same actor without memory, with memory, and with LEXP. The memory condition retains previous observations, actions and intention notes, and execution outcomes. LEXP additionally uses task-level planning and experience-derived feedback as described in Sections 3.3 and 3.4. Model weights and the action interface remain fixed across conditions.

Protocol and metrics. For each task, we conduct three trials. Trials across comparison conditions use similar distributions of initial configurations. Retained experience is initialized according to the evaluated condition, with any experience carried over from earlier attempts explicitly identified. Each trial is limited to 100 actor decisions and ends when the actor signals completion, or when the budget is exhausted. Task success is assessed by human annotations using predefined physical completion criteria, independently of the actor's completion declaration. We report success as the number of successful trials out of three. Task-specific initialization procedures and success criteria are provided in Appendix A.

4.2 Q1: Does LEXP improve performance on real world manipulation tasks?

To answer Q1, we compare vanilla Astra, Astra with memory, and Astra equipped with LEXP across 12 real-world manipulation tasks. As shown in Figure 6a, LEXP achieves 23/36 successful trials (63.9%), compared with 19/36 (52.8%) with memory and 2/36 (5.6%) without memory. Retaining interaction history substantially improves observed performance, while LEXP provides a further aggregate gain of 11.1 percentage points over memory alone. Complete task-level results, including the other evaluated models, are reported in Appendix D, Table 2.

Figure 6c shows that the additional benefit over memory is concentrated in long-horizon execution, where success increases from 3/12 to 7/12. Tool-use and force-control tasks also show higher aggregate success, while fine-manipulation and spatial-reasoning performance remains unchanged in aggregate. Bimanual coordination remains challenging, with no successful sweeping or towel-folding trials under any condition. The following analyses examine how planning, feedback, and retained experience influence execution.

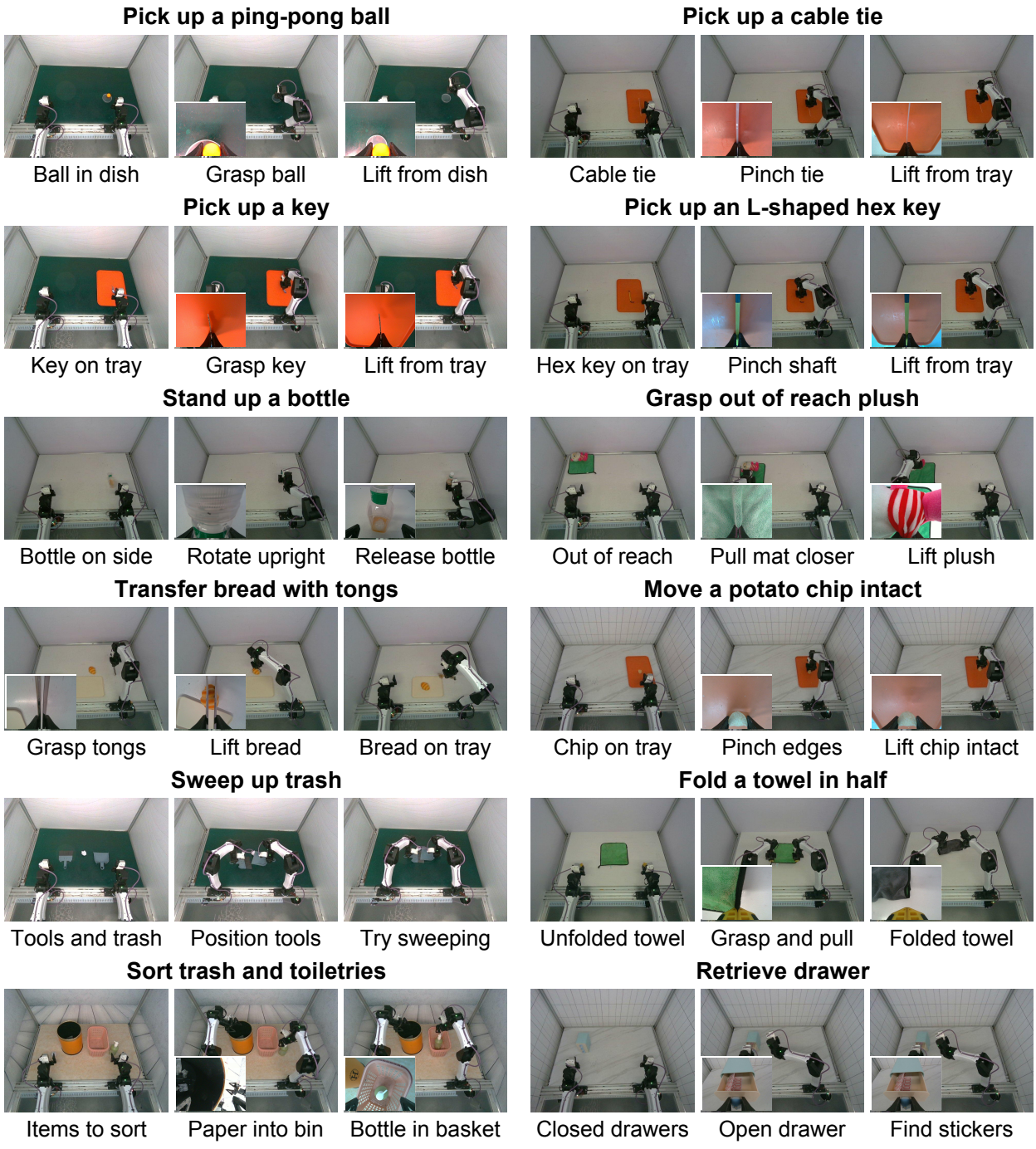


Figure 5 Overview of the 12 real-world manipulation tasks. Each task is illustrated by three selected keyframes showing its initial configuration and representative physical interactions. Main images show top-camera observations; insets provide wrist-camera views of grasping, tool use, and object placement. The sequences illustrate task content and execution stages; quantitative performance is reported separately. Task instructions and completion criteria are provided in Appendix A.

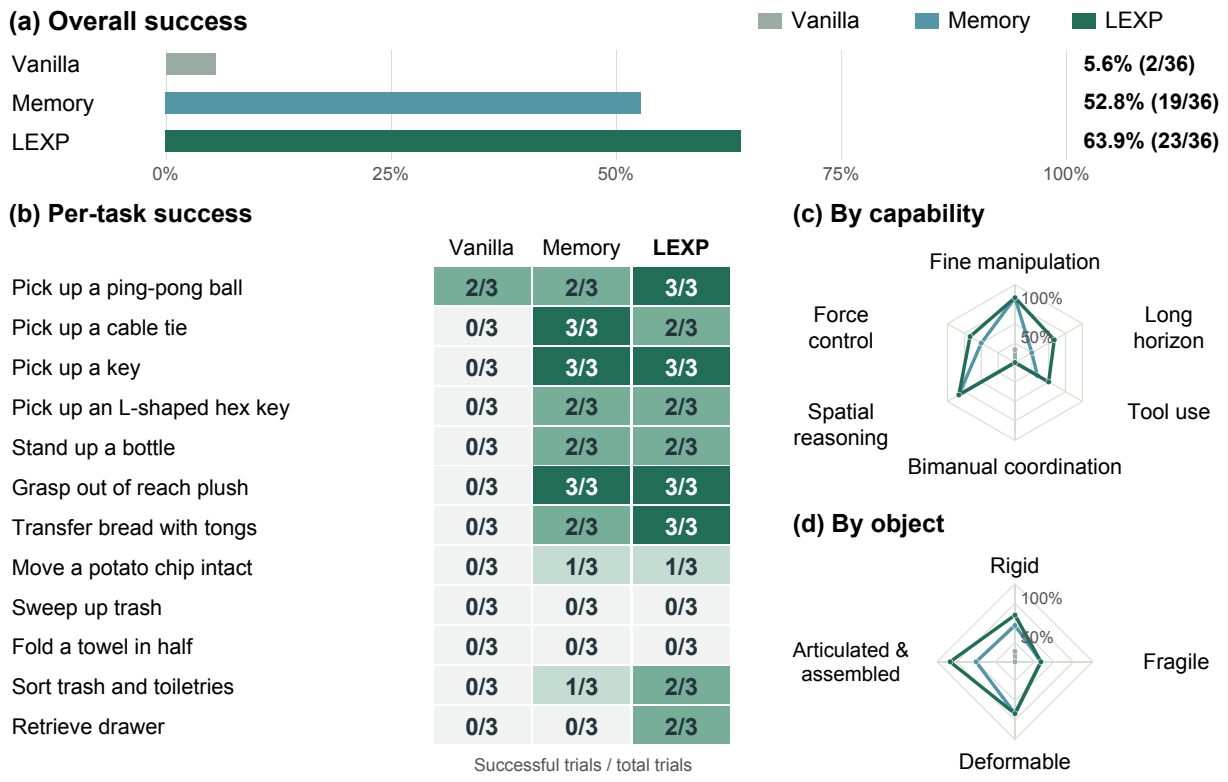


Figure 6 Overall performance comparison. We compare vanilla Astra, Astra with memory, and Astra equipped with LEXP. (a) Overall success across 12 tasks; labels indicate successful trials over total trials. (b) Per-task success; each cell reports successful trials out of three, with darker shading indicating higher success. (c) Mean task-success rates by capability. (d) Mean task-success rates by primary object category. Capability categories may overlap, while each task belongs to one primary object category. Each task is evaluated in three trials per condition. Complete task-level results are reported in Table 2.

4.3 Q2: How does task-level planning guide the actor's reasoning and task progression?

Planning provides task-level structure for reasoning and execution. To examine its contribution, we remove the planner while retaining memory, the experience module, and experience-derived feedback. Across bread transfer, drawer retrieval, and item sorting, complete LEXP achieves 7/9 successful trials and 24/27 completed subtasks, compared with 4/9 and 16/27 without the planner (Figure 7).

The actor can already reason about how to complete these tasks, but execution requires maintaining task objectives through repeated actions and recovery. Different tasks impose different demands, bread transfer follows a relatively direct acquisition then transfer then placement sequence, whereas sorting requires preserving item and container assignments and drawer search requires tracking inspected drawers and opening constraints. Item sorting illustrates the distinction between initial task reasoning and its sustained use. In the memory-only example, the actor initially assigns the bottle to the trash bin and later releases the paper into the pink basket. Without the planner, experience-derived feedback already provides task-level guidance: before the first action, it asks the actor to establish the sorting rule rather than infer destinations from proximity. The actor subsequently states the correct item and container assignments in its action note. Later, feedback recognizes that the paper is inside the bin and redirects attention to the remaining bottle, demonstrating implicit progress tracking. Although the actor revisits the container choice during the prolonged execution and again selects the basket, bottle placement remains incomplete.

LEXP provides an explicit task-level reference through separate subtasks for placing paper in the orange bin and the bottle in the pink basket. The experience module evaluates execution against these goals. When the

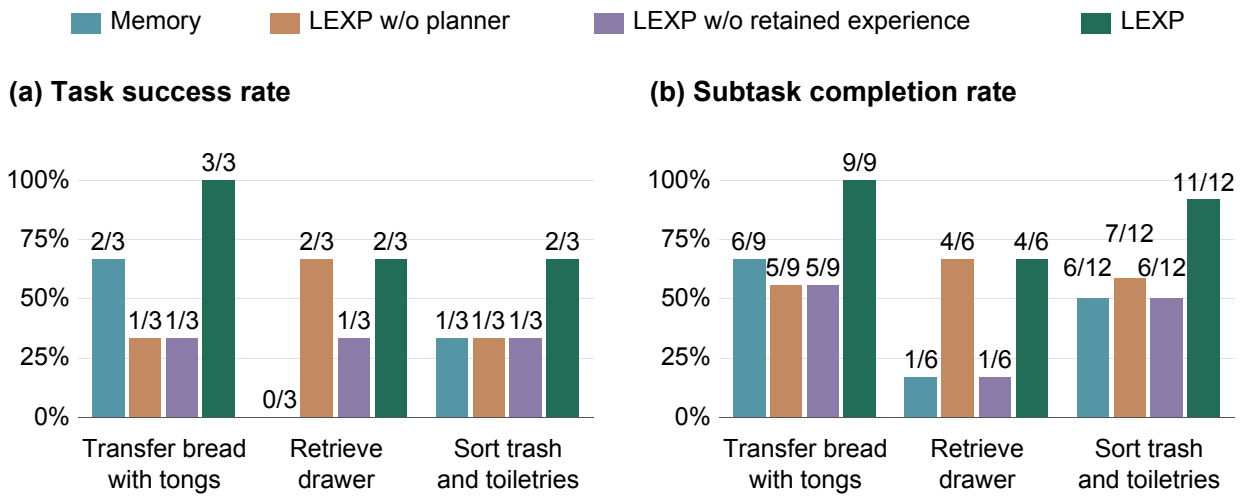


Figure 7 Effects of planning and retained experience on manipulation performance. We compare memory alone, LEXP without the planner, LEXP without retained experience, and complete LEXP on three tasks. (a) Task success rate over three trials per task and condition. (b) Subtask completion rate, calculated as the total number of completed subtasks divided by the total number of evaluated subtasks across the three trials. Labels report successful trials over total trials in (a) and completed subtasks over total subtasks in (b). Both ablated LEXP variants retain experience-derived feedback; the variant without retained experience generates feedback from current observations and recent interactions without access to stored lessons.

paper remains caught on an open fingertip, feedback directs release recovery over the bin and requires visible separation before counting placement as complete. Once paper placement is confirmed, execution proceeds to the bottle, which is subsequently released inside the basket. These traces show that experience feedback can provide task-level guidance on its own, while explicit planning structures this guidance around identified subtasks and their observed completion.

4.4 Q3: Can lessons extracted from physical interaction guide later decisions beyond the interactions from which they were learned?

Physical experience becomes reusable guidance in context space. To examine the contribution of retained experience, we compare complete LEXP with a variant that retains planning and immediate feedback but does not retain or access stored lessons. Across the three evaluated tasks, complete LEXP achieves 7/9 successful trials and 24/27 completed subtasks, compared with 3/9 and 12/27 without retained experience (Figure 7). Observed task success is higher with retained experience on all three tasks. This comparison supports the utility of explicit lesson retention and reuse in the evaluated settings.

Figure 9 illustrates how retained experience enters a later execution during bread transfer. In an earlier episode, repeated lift checks leave the robot uncertain whether the bread is securely held and clear of the supporting surface. At calls 59–60, it moves the empty wrist to seek another viewing angle. Experience accumulated from prior interactions, including this episode, retains an inspection strategy: when tong tips are cropped or contact is uncertain, obtain a view containing the free tips, bread, and supporting surface while keeping the tool hand steady. In a later episode, this retained experience informs guidance to obtain an oblique inspection view before call 5. The guidance enters the actor's context, and the actor subsequently moves the empty wrist camera while maintaining the tool-hand pose and grip.

4.5 Q4: How does experience-derived feedback shape the actor's action decisions?

To answer Q4, we compare memory-only execution with complete LEXP and examine how feedback informs subsequent actions. Across the three tasks in Figure 7, LEXP achieves 7/9 successful trials and 24/27 completed subtasks, compared with 3/9 and 13/27 with memory alone. This comparison establishes the

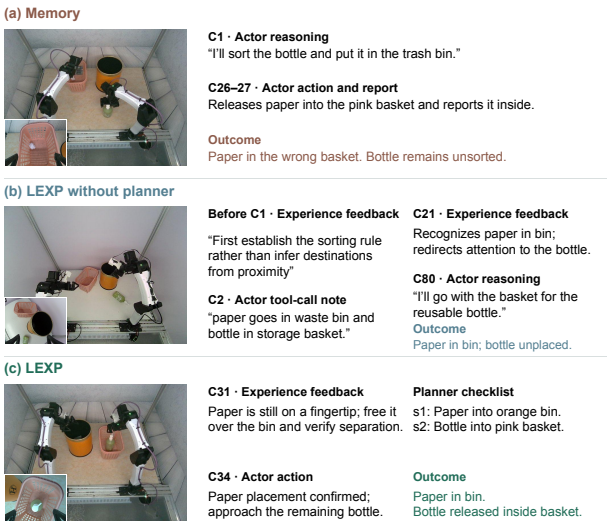


Figure 8 Task-level planning in item sorting. Representative reasoning, guidance, and physical outcomes with memory, LEXP without the planner, and with complete LEXP.

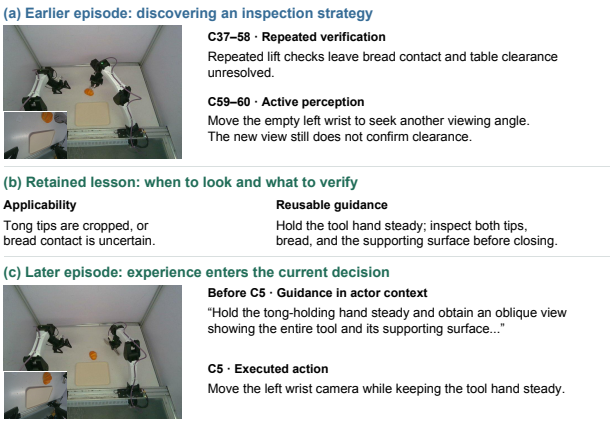


Figure 9 Physical experience becomes reusable guidance in context space. (a) In an earlier bread-transfer episode, uncertain tool contact and clearance motivate an inspection movement using the free wrist camera. (b) Retained experience specifies when additional inspection is needed and what the new view should reveal. (c) In a later episode, this experience informs feedback that guides an inspection-camera movement.

performance benefit of the complete harness; execution traces reveal how task structure and retained experience reach the actor through contextual guidance.

The sorting and bread-transfer examples illustrate two forms of feedback uptake. In sorting (Figure 8), the experience module identifies that paper remains caught on a fingertip despite the jaws being open. Its feedback directs an orientation adjustment and requires visible separation before counting placement as complete. The actor subsequently adjusts the wrist, and later observations confirm the paper inside the bin, separate from the fingers. In bread transfer (Figure 9), retained inspection experience informs guidance to hold the tool hand steady and obtain an oblique view. The actor then moves the empty wrist camera while preserving the tool-hand pose and grip. Together, these traces show feedback guiding both physical recovery and information-gathering actions, translating task-level objectives and prior experience into changes in the actor's subsequent behavior.

5 Related work

5.1 Foundation Models for Robotic Manipulation

Foundation models support robotic manipulation through different interfaces between perception, reasoning, and action. Robot-specialized models such as OpenVLA [10] and $\pi_{0.5}$ [22] learn action generation from robot data, while DreamZero [29] and Cosmos Policy [11] connect visual prediction with action generation. These approaches encode manipulation capabilities through model training. A complementary direction uses general-purpose language and multimodal models to reason over tasks and invoke an external action interface. Here, the central challenge is to connect semantic reasoning with physically feasible execution. Different interfaces address this challenge at different levels. SayCan [2] grounds language-based task planning in the affordances of available skills, while Code as Policies [13] expresses reasoning through executable programs that call perception and control APIs. VoxPoser [8] translates language and visual observations into spatial value maps for motion generation, and ReKep [9] represents manipulation objectives through relational keypoint constraints. Together, these methods show how pretrained reasoning can direct physical behavior through skills, programs, and geometric representations. They also highlight the importance of the surrounding execution system: a model's reasoning must be translated into actions whose consequences can be observed and checked. Recent work increasingly examines this interaction between model capability and

execution scaffolding. CaPX [6] studies code-based manipulation with structured execution feedback and skill synthesis, while concurrent evaluations investigate general-purpose multimodal reasoners as robot policies [4, 19, 26, 33]. Our preliminary evaluation follows this direction by examining fine manipulation, spatial reasoning, and responsiveness to context across real-world tasks. Building on these capabilities, LEXP studies how evidence from physical execution can guide subsequent decisions while keeping both model parameters and the action interface fixed.

5.2 In-context Robot Learning

In-context robot learning allows behavior to depend on examples supplied at deployment. ICRT [5] conditions action prediction on sensorimotor trajectories, enabling demonstrations to specify new behaviors without task-specific parameter updates. RICL [25] develops in-context learning capabilities in a pretrained robot policy and retrieves relevant demonstration segments at deployment. RoboPrompt [30] instead represents demonstrations through textual descriptions of object poses and end-effector actions for an off-the-shelf language model. These approaches demonstrate that contextual examples can communicate task-relevant behavior, although they differ in the representation of demonstrations and the training required to make those examples usable. Feedback extends context conditioning beyond examples supplied before execution. Inner Monologue [7] incorporates scene descriptions, success feedback, and human input into closed-loop planning, while InCoRo [36] investigates in-context control with interaction feedback. Language corrections can also communicate changes that are difficult to specify through a scalar success signal: interactive corrections [23] and Language Model Predictive Control [14] use such information to revise robot behavior. These methods establish context as an interface for changing plans and actions during execution. Our preliminary demonstration and language-feedback experiments examine this interface in the evaluated manipulation setting. A further question is how corrective information is obtained and retained. REFLECT [15] uses multisensory execution summaries to explain failures and support corrective planning. DROC [31] distills human corrections into reusable knowledge and retrieves relevant corrections for subsequent tasks and object instances. These approaches connect immediate feedback with more persistent behavioral guidance. LEXP develops this connection using the robot's own physical interactions: raw history records what happened, while an experience module extracts lessons, retains their applicability conditions and limitations, and interprets them in relation to the current subtask. The resulting feedback brings relevant experience into the actor's context.

5.3 Learning from Experience and Context Adaptation

Physical interaction can improve robot behavior through updates to policies, skills, or external knowledge. Reinforcement-learning systems such as SERL [17] and HIL-SERL [18] improve policies using real-world interaction, with human intervention supporting data collection and recovery. $\pi_{0.6}$ [21] likewise studies improvement from experience for generalist robot policies. These approaches update model parameters to incorporate interaction evidence. LEXP pursues improvement through a different adaptation variable: the contextual guidance supplied to a frozen actor. Experience can also modify the structures surrounding a policy. ReSYNC [12] connects failure recovery with the acquisition of relational concepts for abstract planning. ASPIRE [16] refines reusable robot skills through execution evidence, while ENPIRE [28] organizes physical policy improvement through an agentic experimentation loop. More recent systems extend adaptation to the execution harness itself. SHAPER [27] evolves reusable skills and a context-code harness with frozen model parameters, and Harness VLA [34] uses execution memory to guide the deployment of fixed manipulation primitives. These works show that experience-driven improvement can operate at several levels beyond policy weights, including programs, abstractions, skills, and contextual knowledge. Related developments in language agents provide mechanisms for retaining and applying experience in textual form. Reflexion [24] uses verbal reflections as memory for subsequent attempts, and ExpeL [35] extracts reusable insights from accumulated experiences. GEPA [1] uses execution traces and reflection to optimize prompts, while Training-Free GRPO [3] distills experiential knowledge into contextual guidance. ACE [32] further organizes adaptation through the incremental generation, reflection, and curation of contextual knowledge. These methods motivate treating context as a persistent adaptation mechanism rather than only a record of the current interaction. Applying this principle to manipulation requires grounding retained guidance in physical evidence. An intended action may not execute as proposed, an executed motion may not achieve its intended effect, and a previously

useful correction may depend on object pose, contact state, or task progress. LEXP therefore connects task-level planning with evidence-based experience extraction and reuse. The planner supplies the current task structure; the experience module interprets recent interactions and retained lessons against the observed scene, then generates feedback for the actor. Our evaluation examines how this organization affects real-world manipulation performance, task progression, and the reuse of interaction-derived guidance.

6 Conclusion

We propose LEXP, a harness for learning from physical experience through context-space adaptation. LEXP combines task-level planning with the extraction and reuse of interaction-derived lessons, translating physical evidence into contextual feedback without updating model weights. Across 12 real-world manipulation tasks, LEXP achieves 23/36 successful trials (63.9%), compared with 19/36 (52.8%) with memory alone and 2/36 (5.6%) for vanilla Astra. On three tasks, removing planning reduces success from 7/9 to 4/9, while removing retained experience reduces it to 3/9. Execution analyses illustrate how feedback guides action correction and how retained experience informs a later episode. These results highlight the potential of physical experience as reusable guidance for robotic manipulation.

References

[1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. arXiv preprint arXiv:2507.19457, 2025. doi: 10.48550/arXiv.2507.19457. URL https://arxiv.org/abs/2507.19457.

[2] Michael Ahn, Anthony Brohan, Noah Brown, et al. Do as i can, not as i say: Grounding language in robotic affordances. arXiv preprint arXiv:2204.01691, 2022. doi: 10.48550/arXiv.2204.01691. URL https://arxiv.org/abs/2204.01691.

[3] Yuzheng Cai, Siqi Cai, Yuchen Shi, Zihan Xu, Lichao Chen, Yulei Qin, Xiaoyu Tan, Gang Li, Zongyi Li, Haojia Lin, Yong Mao, Ke Li, and Xing Sun. Training-free group relative policy optimization. arXiv preprint arXiv:2510.08191, 2025. doi: 10.48550/arXiv.2510.08191. URL https://arxiv.org/abs/2510.08191.

[4] Dongzhou Cheng, Taoran Yi, Ye Fang, Xingwu Zhang, Fan Feng, Yixuan Li, Gengxiong Zhuang, Rongze Wang, Shuai Yang, Wei Song, Weizhi Xue, Minyan Wu, Jie Gui, Jiaqi Wang, and Tong Wu. In-context robot learning with VLM agents. arXiv preprint arXiv:2609.19138, 2026. doi: 10.48550/arXiv.2609.19138. URL https://arxiv.org/abs/2609.19138.

[5] Letian Fu, Huang Huang, Gaurav Datta, Lawrence Yunliang Chen, William Chung-Ho Panitch, Fangchen Liu, Hui Li, and Ken Goldberg. In-context imitation learning via next-token prediction. arXiv preprint arXiv:2408.15980, 2024. doi: 10.48550/arXiv.2408.15980. URL https://arxiv.org/abs/2408.15980.

[6] Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Guanzhi Wang, Dantong Niu, Fei-Fei Li, Guanya Shi, Jiajun Wu, Shankar Sastry, Yuke Zhu, Ken Goldberg, and Linxi Fan. CaP-X: A framework for benchmarking and improving coding agents for robot manipulation. arXiv preprint arXiv:2603.22435, 2026. doi: 10.48550/arXiv.2603.22435. URL https://arxiv.org/abs/2603.22435v2. Initial submission March 23, 2026; version 2 July 2, 2026.

[7] Wenlong Huang, Fei Xia, Ted Xiao, et al. Inner monologue: Embodied reasoning through planning with language models. arXiv preprint arXiv:2207.05608, 2022. doi: 10.48550/arXiv.2207.05608. URL https://arxiv.org/abs/2207.05608.

[8] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. VoxPoser: Composable 3d value maps for robotic manipulation with language models. arXiv preprint arXiv:2307.05973, 2023. doi: 10.48550/arXiv.2307.05973. URL https://arxiv.org/abs/2307.05973.

[9] Wenlong Huang, Chen Wang, Yunzhu Li, Ruohan Zhang, and Li Fei-Fei. ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation. arXiv preprint arXiv:2409.01652, 2024. doi: 10.48550/arXiv.2409.01652. URL https://arxiv.org/abs/2409.01652.

[10] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. arXiv preprint arXiv:2406.09246, 2024. doi: 10.48550/arXiv.2406.09246. URL https://arxiv.org/abs/2406.09246.

[11] Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, and Jinwei Gu. Cosmos policy: Fine-tuning video models for visuomotor control and planning. arXiv preprint arXiv:2601.16163, 2026. doi: 10.48550/arXiv.2601.16163. URL https://arxiv.org/abs/2601.16163.

[12] Bowen Li, Mayank Mishra, Y. Isabel Liu, Stone Tao, Nishanth Kumar, Alexander G. Gray, Ruwan Wickramarachchi, Jonathan Francis, Sebastian Scherer, and Tom Silver. Recover, discover, plan: Learning skills and concepts from robot failures. arXiv preprint arXiv:2606.18328, 2026. doi: 10.48550/arXiv.2606.18328. URL https://arxiv.org/abs/2606.18328.

[13] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. arXiv preprint arXiv:2209.07753, 2022. doi: 10.48550/arXiv.2209.07753. URL https://arxiv.org/abs/2209.07753.

[14] Jacky Liang, Fei Xia, Wenhao Yu, et al. Learning to learn faster from human feedback with language model predictive control. arXiv preprint arXiv:2402.11450, 2024. doi: 10.48550/arXiv.2402.11450. URL https://arxiv.org/abs/2402.11450.

[15] Zeyi Liu, Arpit Bahety, and Shuran Song. REFLECT: Summarizing robot experiences for failure explanation and correction. arXiv preprint arXiv:2306.15724, 2023. doi: 10.48550/arXiv.2306.15724. URL https://arxiv.org/abs/2306.15724.

[16] Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, Mosharaf Chowdhury, Yuke Zhu, Linxi Fan, and Guanzhi Wang. ASPIRE: Agentic /skills discovery for robotics. arXiv preprint arXiv:2607.00272, 2026. doi: 10.48550/arXiv.2607.00272. URL https://arxiv.org/abs/2607.00272. Initial submission June 30, 2026.

[17] Jianlan Luo, Zheyuan Hu, Charles Xu, You Liang Tan, Jacob Berg, Archit Sharma, Stefan Schaal, Chelsea Finn, Abhishek Gupta, and Sergey Levine. SERL: A software suite for sample-efficient robotic reinforcement learning. arXiv preprint arXiv:2401.16013, 2024. doi: 10.48550/arXiv.2401.16013. URL https://arxiv.org/abs/2401.16013.

[18] Jianlan Luo, Charles Xu, Jeffrey Wu, and Sergey Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. Science Robotics, 10(105):eads5033, 2025. doi: 10.1126/scirobotics.ads5033. URL https://doi.org/10.1126/scirobotics.ads5033.

[19] Achu Menon, Sravanthi Machcha, Sabrina Zou, Tzu Kit Chan, and Jay Chooi. GPT-6 Astra on robotic manipulation. RoboCurve evaluation report, September 4, 2026. URL https://openai.robocurve.org/gpt-6-astra/. Accessed September 25, 2026.

[20] OpenAI. GPT-6 Astra model. OpenAI API documentation, 2026. URL https://developers.openai.com/api/docs/models/gpt-6-astra. Accessed September 26, 2026.

[21] Physical Intelligence, Ali Amin, Raichelle Aniceto, et al. $\pi^*_{0.6}$: a VLA that learns from experience. arXiv preprint arXiv:2511.14759, 2025. doi: 10.48550/arXiv.2511.14759. URL https://arxiv.org/abs/2511.14759.

[22] Physical Intelligence, Kevin Black, Noah Brown, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. arXiv preprint arXiv:2504.16054, 2025. doi: 10.48550/arXiv.2504.16054. URL https://arxiv.org/abs/2504.16054.

[23] Pratyusha Sharma, Balakumar Sundaralingam, Valts Blukis, Chris Paxton, Tucker Hermans, Antonio Torralba, Jacob Andreas, and Dieter Fox. Correcting robot plans with natural language feedback. arXiv preprint arXiv:2204.05186, 2022. doi: 10.48550/arXiv.2204.05186. URL https://arxiv.org/abs/2204.05186.

[24] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. arXiv preprint arXiv:2303.11366, 2023. doi: 10.48550/arXiv.2303.11366. URL https://arxiv.org/abs/2303.11366.

[25] Kaustubh Sridhar, Souradeep Dutta, Dinesh Jayaraman, and Insup Lee. RICL: Adding in-context adaptability to pre-trained vision-language-action models. arXiv preprint arXiv:2508.02062, 2025. doi: 10.48550/arXiv.2508.02062. URL https://arxiv.org/abs/2508.02062.

[26] Jiayi Su, Yixin Zheng, Mi Yan, Li Yi, Zhizheng Zhang, and He Wang. GPT 6 Astra as an embodied policy. Technical report and code, 2026. URL https://github.com/anonymous-report-421/GPT-as-Policy. Accessed September 25, 2026.

[27] Peidong Wang, Zhiming Ma, Ying Chang, Xufang Luo, Yiqun Zhang, Zihan Wang, Xiaocui Yang, Shi Feng, Yuqing Yang, and Dongsheng Li. Self-evolving embodied agents via skill-harness evolution. arXiv preprint arXiv:2608.11350, 2026. doi: 10.48550/arXiv.2608.11350. URL https://arxiv.org/abs/2608.11350v2. Initial submission August 11, 2026; version 2 September 10, 2026.

[28] Wenli Xiao, Jia Xie, Tonghe Zhang, Haotian Lin, Letian Fu, Haoru Xue, Jalen Lu, Yi Yang, Cunxi Dai, Zi Wang, Jimmy Wu, Guanzhi Wang, S. Shankar Sastry, Ken Goldberg, Linxi Fan, Yuke Zhu, and Guanya Shi. ENPIRE: Agentic robot policy self-improvement in the real world. arXiv preprint arXiv:2606.19980, 2026. doi: 10.48550/arXiv.2606.19980. URL https://arxiv.org/abs/2606.19980v2. Initial submission June 18, 2026; version 2 September 20, 2026.

[29] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, et al. World action models are zero-shot policies. arXiv preprint arXiv:2602.15922, 2026. doi: 10.48550/arXiv.2602.15922. URL https://arxiv.org/abs/2602.15922.

[30] Yida Yin, Zekai Wang, Yuvan Sharma, Dantong Niu, Trevor Darrell, and Roei Herzig. In-context learning enables robot action prediction in LLMs. arXiv preprint arXiv:2410.12782, 2024. doi: 10.48550/arXiv.2410.12782. URL https://arxiv.org/abs/2410.12782.

[31] Lihan Zha, Yuchen Cui, Li-Heng Lin, Minae Kwon, Montserrat Gonzalez Arenas, Andy Zeng, Fei Xia, and Dorsa Sadigh. Distilling and retrieving generalizable knowledge for robot manipulation via language corrections. arXiv preprint arXiv:2311.10678, 2023. doi: 10.48550/arXiv.2311.10678. URL https://arxiv.org/abs/2311.10678.

[32] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. arXiv preprint arXiv:2510.04618, 2025. doi: 10.48550/arXiv.2510.04618. URL https://arxiv.org/abs/2510.04618.

[33] Wenbo Zhang, Kaixuan Wang, Yutao Ouyang, Xiaoyu Huang, Liyang Li, Kailun Su, Weiyang Jin, Wenhao Chai, Haotian Liang, Zhiyang Dou, Yue Chen, and Tianxing Chen. An unexpected robot policy: Early evaluations of GPT-6 Astra on RoboDojo and beyond. arXiv preprint arXiv:2609.24170, 2026. doi: 10.48550/arXiv.2609.24170. URL https://arxiv.org/abs/2609.24170.

[34] Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Yi Nie, Chunyang Zhu, Jiaxing Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, Jiaji Rao, Zhengru Fang, Changxu Wei, Yu Wang, Wenbo Ding, and Chao Yu. Harness VLA: Steering frozen VLAs into reliable manipulation primitives via memory-guided agents. arXiv preprint arXiv:2607.08448, 2026. doi: 10.48550/arXiv.2607.08448. URL https://arxiv.org/abs/2607.08448v5. Version 5, September 24, 2026.

[35] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. arXiv preprint arXiv:2308.10144, 2023. doi: 10.48550/arXiv.2308.10144. URL https://arxiv.org/abs/2308.10144.

[36] Jiaqiang Ye Zhu, Carla Gomez Cano, David Vazquez Bermudez, and Michal Drozdzal. InCoRo: In-context learning for robotics control with feedback loops. arXiv preprint arXiv:2402.05188, 2024. doi: 10.48550/arXiv.2402.05188. URL https://arxiv.org/abs/2402.05188.

Appendix

A Task Details

Table 1 specifies the instructions, initial configurations, and physical success criteria for the 12 evaluated tasks. Trials across comparison conditions use similar distributions of initial configurations. Task success is determined by human annotation of the observed physical result according to these criteria. Where intermediate progress is reported, subgoals are defined independently of the planner's generated checklist.

Table 1 Task instructions, initial configurations, and physical success criteria for the 12 real-world manipulation tasks.

Task	Instruction	Initial configuration	Success criterion
Pick up a ping-pong ball	Pick up the ball.	A ping-pong ball rests inside a shallow transparent dish on the tabletop; both arms start parked clear of the dish.	The ball is enclosed between the fingertips and held clear of the dish, which remains on the tabletop.
Pick up a cable tie	Pick up the cable tie.	A white cable tie lies on an orange tray on the tabletop.	The cable tie is retained between the fingertips, with its free length extending from the raised gripper clear of the tray.
Pick up a key	Pick up the key.	A key on a split ring with an attached tag lies on the table.	The key, ring, and tag are suspended from the raised gripper clear of the surface; the grasp may be on the key, ring, or tag.
Pick up an L-shaped hex key	Pick up the hex key.	An L-shaped hex key lies on the table with its long leg and bent end accessible.	The hex-key shaft is pinched between the opposing gripper pads, with its bent end projecting clear of the work surface.
Stand up a bottle	Stand the bottle upright.	The bottle lies on its side on the tabletop.	The bottle stands cap-up on its base on the tabletop, with visible clearance from both grippers and no robot support.
Grasp out of reach plush	Pick up the plush toy.	A plush toy rests on a mat positioned beyond the arm's direct reach.	Plush fabric is retained between the fingertips, and the toy hangs from the raised gripper clear of the mat, which remains flat below.
Transfer bread with tongs	Use the tong to pick up all the bread and put on the tray.	Tongs rest on the tabletop; bread pieces lie on the table behind a tray.	All bread pieces are visible within the tray in the overhead view, while the tongs' working ends are separated, empty, and clear of the bread.
Move a potato chip intact	Pick up the chip.	A potato chip rests on the tabletop.	The chip is pinched between the fingertips, suspended clear of the surface, and remains unbroken.
Sweep up trash	Sweep the debris into the dustpan.	Debris lies on the work surface; a brush and a dustpan rest nearby, to be held one per arm.	One gripper holds the brush handle and the other holds the dustpan. The dustpan's lip rests against the surface with its mouth facing the debris, and the debris ends up inside the pan.
Fold a towel in half	Fold the towel in half.	A towel lies flat and unfolded on the table.	The towel rests as a doubled rectangle with one half covering the other, with opposite corners and side hems brought together.

Continued on next page

Table 1 (continued)

Task	Instruction	Initial configuration	Success criterion
Sort trash and toiletries	Sort the items on the table into the appropriate containers.	Crumpled paper and a pump bottle lie on the table beside a cylindrical bin and a basket.	The crumpled paper rests inside the bin, and the pump bottle stands on the floor of the basket, both separate from the withdrawn fingers.
Retrieve drawer	Find the drawer with the sticker. Open each drawer at most once.	A three-drawer cabinet with knobs starts with all drawers shut; a sheet of stickers is inside exactly one drawer.	A drawer is open with the sticker sheet visible inside. Each drawer has been opened at most once, and non-target drawers have been pushed shut again.

B Preliminary Context-Steering Setup

The preliminary experiments examine whether externally supplied context can redirect execution and recovery. These inputs are distinct from the agent-generated experience feedback evaluated in LEXP.

B.1 Visual Demonstrations

A visual demonstration is a human-recorded video showing how a task is performed. It provides an execution reference, including where present, recovery from unsuccessful interactions. We process each recording into candidate frames, from which the model selects frames relevant to the task. The selected frames are inserted into the actor's context as visual references, alongside observations of the robot's current scene. They condition the actor's decisions without directly specifying robot motion commands.

Specifically, frame selection is performed through a separate, stateless model request containing the task instruction and timestamped candidate images. The model selects a fixed number of frames, including the first and last frames, and provides a relevance explanation for each selection. The frame-selection prompt and the actor's demonstration-input format are shown below.

Demonstration frame-selection system prompt

You are preparing to perform a robot manipulation task. Select demonstration frames most useful for completing the task with the robot: task stages, object relationships and before/after effects. Return frame IDs and a short explanation of each frame's relevance. No robot movement is allowed in this phase.

Frame-selection input template

TASK: <task instruction>
DEMONSTRATION TYPE: <demonstration type>
Select exactly <number of selected frames> frames, including endpoints <first frame ID> and <last frame ID>.
frame <candidate frame ID>, t=<timestamp>s
Camera: video
<candidate image; repeated for each candidate>

After selected frames, for human demonstrations, the actor receives the following context block.

Visual demonstration in the actor context

<HUMAN or RECOVERY> VIDEO DEMONSTRATION: use these selected frames as task references. The demonstrated embodiment and viewpoint may differ. No robot states are supplied in this block. No actions either.
human demo frame <frame ID>, t=<timestamp>s
<selected image; repeated for each selected frame>

In the towel-folding comparison, clean and recovery recordings provide different execution references; the resulting behavioral differences are reported in Section 2.2.

B.2 Online Language Feedback

Online language feedback consists of human-provided textual guidance during robot execution. The feedback is inserted into the actor's context before a subsequent decision, allowing the actor to interpret it together with current observations and generate the corresponding tool call.

Pending messages are inserted into the actor's context using the following wrapper:

Online language feedback in the actor context

OPERATOR FEEDBACK:
<human message>
<additional pending messages, if any>

The actor interprets the message together with its current observations and produces its own tool call.

The barbecue-transfer analysis in Section 2.2 relates delivered messages to subsequent changes in motion direction, clearance, arm selection, and action order.

C LEXP Implementation Details

C.1 Experience Representation and Updates

An experience entry contains:

- `lesson`: what was attempted or observed and what followed;
- `applicability`: when the finding applies and what would invalidate it;
- `feedback_use`: which later decision the finding can inform;
- `limits_and_counterevidence`: what the evidence does not establish;
- `unresolved`: alternative explanations or simultaneous changes that remain unresolved;
- `evidence_ids`: supporting interaction references;
- `revises`: the prior entry replaced by an update, where applicable.

The applicability of a retained lesson is distinct from the condition attached to a proposed method. The former concerns reuse of a past finding; the latter concerns whether a current proposal is appropriate.

Experience updates are generated alongside contextual feedback. Each update either creates a new entry or identifies an existing entry through `revises`. A revision supersedes the referenced entry in the active experience list while preserving its provenance. The resulting list is supplied to subsequent feedback calls. When cross-episode seeding is enabled, findings from earlier attempts are selected and compressed to initialize the new episode's list.

D Complete Experimental Results

Table 2 reports task-level success counts for all evaluated models and conditions.

Table 2 Complete task-success results. Entries report successful trials over total trials. Overall results use all 12 tasks, with three trials per task and condition.

	GPT-6 Astra			Opus 5.5	Sol 5.6
Task	Vanilla	Memory	LEXP	Memory	Memory
Pick up a ping-pong ball	2/3	2/3	**3/3**	1/3	0/3
Pick up a cable tie	0/3	**3/3**	2/3	1/3	0/3
Pick up a key	0/3	**3/3**	**3/3**	1/3	0/3
Pick up an L-shaped hex key	0/3	**2/3**	**2/3**	**2/3**	0/3
Stand up a bottle	0/3	**2/3**	**2/3**	0/3	0/3
Grasp out of reach plush	0/3	**3/3**	**3/3**	**3/3**	0/3
Transfer bread with tongs	0/3	2/3	**3/3**	1/3	0/3
Move a potato chip intact	0/3	**1/3**	**1/3**	0/3	0/3
Sweep up trash	0/3	0/3	0/3	0/3	0/3
Fold a towel in half	0/3	0/3	0/3	0/3	0/3
Sort trash and toiletries	0/3	1/3	**2/3**	0/3	0/3
Retrieve drawer	0/3	0/3	**2/3**	**2/3**	0/3
Overall (12 tasks)	2/36	19/36	**23/36**	11/36	0/36
Success rate (%)	5.6	52.8	**63.9**	30.6	0.0

E Prompts and Structured Outputs

We provide the prompts and structured-output specifications used for the evaluated configurations.

E.1 Actor

The actor system prompt is assembled from embodiment and coordinate conventions, the action contract, configured movement limits, station geometry when supplied, and the verification, retry, and termination instructions. Task instructions, experience feedback, memory, current state and images, and the remaining decision budget are supplied as content. Planner methods and retained lessons reach the actor through experience feedback.

System-prompt template

You control a physical robot through one tool call per decision. Observations are measured inputs; action outcomes and task success are distinct. Joint angles are radians; positions are metres in each arm's base frame. Gripper opening is normalized: 0 closed, 1 open. Camera order: <camera_roles>.
<factual_action_contract>
Configured limits: <movement_limits>
<station_geometry>
<verification_instruction>
<retry_instruction>
<termination_instruction>

Verify instruction

After execution, compare the new observation with the intended effect. An accepted command does not establish task success.

Retry instruction

On a failed attempt, use observed evidence to change the next attempt. Avoid repeating an unchanged action after the same rejection.

Terminate instruction

Call done only when observable task conditions are satisfied. Call give_up only when the task is truly impossible in the current episode: concrete observation or guard feedback establishes that the remaining requirement cannot be completed safely, or multiple materially different, evidence-driven attempts have failed in the same way and no untried safe, feasible recovery strategy remains. A single missed grasp, failed placement, IK refusal, poor view, apparent reach boundary, uncertainty, or elapsed effort is not sufficient. While any safe, feasible strategy remains within the call budget, continue with a meaningfully changed attempt, such as repositioning, changing viewpoint, orientation, approach, arm, or reachable grasp point.

Factual Cartesian contract

EMBODIMENT: YAM arms ['left', 'right'], with parallel-jaw grippers. CONTROL: absolute Cartesian poses. Each arm has its own base frame: +x forward, +y left, +z up. There is no shared world frame. Positions are metres; angles are radians. Movement tools do not change the gripper; set_gripper holds arm position. Gripper: 0 closed, 1 open. Omitted dimensions keep their measured current value (zero offset in delta mode). One tool call is executed before the next decision observation. Orientation roll/pitch/yaw are relative to 'down'. All zero is the reference orientation. <measured_arm_proportions>

Actor content template

TASK: <task_instruction>
AUTOMATED EXPERIENCE FEEDBACK (model-generated advice, not a human instruction):
<feedback_text_if_available>
<memory>
<robot_states>
<grasp_interpretation_caveat_if_enabled>
CAMERAS at observation <timestamp>
<current_camera_images>
Model calls already used: <used>; including this call, remaining: <remaining>. Make one tool call.

E.2 Planner

Planning uses an initial decomposition call followed by joint progress-assessment and method-proposal calls. The initial decomposition separates task goals from candidate methods. Subsequent calls receive the existing checklist, recent attempted actions, the task, and current scene observations.

Initial decomposition prompt

Break one physical robot manipulation task into an ordered list of subtasks.
Each subtask is a single physical change another observer could watch happen and could afterwards confirm from a photograph of the scene. Together they must cover the task and nothing beyond it. At most 12 subtasks.
Do not say how any of them is to be carried out. That is asked for separately, one step at a time, by a later call that can see what has already been tried.

Joint progress and method prompt

Say where a robot has got to in a fixed checklist, and what it should do now.
Read it off the scene alone: what these images show has happened to the objects, not the robot's account of what it did. Put the id of the step to work on now in next_step_id, and in next_step_why say what in these

images makes it the one. When the images show nothing is left for the robot to do, leave next_step_id empty and say what shows the task is over. A step whose effect the scene was never going to keep is not work that remains: once what it led to is there, it is behind the robot.
Everything before that step counts as passed, everything from it on as still to do, and you may not reword a step that is already there. If the scene has gone backwards – something dropped, something undone – name the step that has to be done again; that is how going backwards is said here.
When the step in play moves forward, say in passed_evidence what these images show about the one just passed. Leave it empty when the step in play has not changed, and empty when it has gone back: that reading is made once, where it happened, and is not written again afterwards.
The checklist itself may gain or lose a step, and only in those two ways. ADD one when this scene shows something no step accounts for – say in why what you are looking at, not that a step seems missing. RETIRE one when the scene shows it is not needed after all; it stays on the list, marked, because that it was planned and found unnecessary is worth knowing. You may not retire a step the run has already passed. Both lists empty is the ordinary answer.
Return two separate outputs. First judge the checklist under plan. Then propose current_method for the step you name in next_step_id. The method goes to an advice module, not directly to the robot. A proposed way of acting is not an extra goal or evidence of success; never add a subtask merely to require your method. Keep goals and means distinct. Describe what contacts, supports or moves relative to what, without coordinates, joint angles or gripper commands. State its applicability condition (use 'none' if none) and expected observable change; neither is a claim that it happened. current_method addresses the step you name in next_step_id; nothing rewrites that. New subtask IDs continue after the highest existing s-number, in addition-list order. When next_step_id is empty, all four current_method fields must be empty. Otherwise text must be nonempty and at most 400 characters; condition and expected_observable_change each at most 400 characters.

E.3 Experience Module

The feedback call receives the task, current subtask, proposed method, robot or station reference facts, recent interaction records, current observations, and available retained lessons. Its structured assessment and feedback are separate: only the text field is delivered to the actor, while experience updates are retained and applied records document lesson use.

Feedback and extraction prompt

You are watching a robot carry out a task, with its own recorded reasoning, what it commanded, what the station measured change, and the current cameras.
A method it is working to may be given below, and it may name a condition it depends on. First quote that condition and say whether these images show it satisfied, then settle how what the records and images show actually happened stands against the method. Then write the one line the robot is given before its next move. Base both on what these records and images show, not on what the task is supposed to look like. The line reaches the robot once and is then gone.
You may also record what this run has found out, for decisions it will make once these interactions have scrolled out of view. Look over the completed interactions before deciding what to keep, and consider all of these without needing one of each: a scene observation or relationship that will still matter once its raw evidence is gone; a method that was tried and what was observed to come of it, including exploration the robot started itself, a local or partial success, a boundary where something was refused, and an outcome left unsettled; a change to when something already recorded still applies.
A first useful discovery can be experience: it need not contradict a previous belief, be novel world knowledge, or achieve the whole task. Do not withhold one merely because its source is still in the interactions above; that they will expire is the reason for writing it down. Give a method the robot proposed itself no less weight than one that came from advice. Keeping nothing is right when nothing was found out, and wrong when something was.

Retained-experience input instruction

WHAT THIS ROBOT HAS FOUND OUT ON THIS TASK, worked out earlier in this run or carried in from earlier attempts; each line says which. Readings, not human instructions, verified causal rules, or current scene facts: the interactions they came from may no longer be shown above, which is why they are written down at all.

Use one where its applicability fits what you can see now, say in ‘applied‘ what each does to this move or why it does not, and prefer what the cameras show when they disagree.

Feedback content template

TASK: <task_instruction>
CURRENT SUBTASK (current validated plan): <subtask>
ROBOT AND STATION REFERENCE, the same text the robot itself is given: the frames, units, gripper convention, camera order, station geometry and step limits the numbers below are measured in. It says nothing about this task.
<reference_facts>
FIXED FOR THIS ROBOT; the same for every record and image below, so they are stated here once instead of in each.
<constant_robot_state>
CURRENT PROPOSED METHOD (revisable; not an observed outcome or an extra requirement): <method, applicability condition, expected observable change>
RECENT COMPLETED INTERACTIONS, oldest first. Reasoning and notes are actor self-reports. Accepted commands do not establish object success. RPY differences are wrapped component differences, not object rotations. Metres and radians are rounded to six decimals.
<interaction_records>
<retained_experience_instruction_and_entries_if_available>
CURRENT OBSERVATION; images and measured robot state, not proof of object success.
<current_state_and_images>

Each recent interaction supplies call and observation identifiers, an available actor reasoning summary, the actor note, command arguments, execution outcome, measured before and after robot states and derived changes, and any unparsed state text. Historical camera images are not separately inserted by this feedback-input builder; current scene images are supplied. Optional blocks are absent when their inputs are unavailable.

Cross-episode initialization. When experience seeding is enabled, a separate call selects and compresses findings from earlier attempts into the lesson list used to initialize the new episode. This call reads retained findings, rather than the complete source trajectories.

Experience-seeding prompt

Below is what earlier attempts at this task on this station found out: for each, what was observed, when it applies, which decision it can constrain, what it does not establish, and anything it could not tell apart. Write what this episode should start out already knowing. It faces a different arrangement of the same task on the same station, so carry only what will still be true there – an applicability that names this one arrangement cannot be used in another. Keep what each entry does not establish; dropping it is how a local result becomes a rule. Everything you carry costs this episode something to read, and carrying nothing is a legitimate answer.